# Parallel Connected Components

MPI + OpenCilk Hybrid Implementation

Parallel and Distributed Computing

Evangelos Moschou
AEM: 10986

December 14, 2025

**Abstract**

This report presents a hybrid MPI+OpenCilk parallel algorithm for computing connected components in large sparse graphs. We describe the algorithm, key optimizations (asynchronous communication, edge-balanced partitioning), and present performance results showing 2.5x speedup with asynchronous BFS.

## 1 Introduction

Finding connected components in large graphs is a fundamental problem in graph analytics with applications in social network analysis, image processing, and scientific computing. As graph sizes grow to billions of edges, parallel algorithms become essential for practical computation times.

This project implements a parallel connected components algorithm using:

- **MPI** (Message Passing Interface) for distributed memory parallelism across cluster nodes.
- **OpenCilk** for shared memory parallelism within each node using work-stealing.

The input graphs are stored in MATLAB's `.mat` format and converted to Compressed Sparse Row (CSR) format for efficient traversal.

## 2 Algorithm Design

### 2.1 Graph Representation

Graphs are stored in CSR format with `row_ptr[n+1]` (pointers to adjacency lists) and `col_idx[m]` (destination vertices). This provides $O(1)$ neighbor access and efficient cache utilization.

### 2.2 Label Propagation

The algorithm uses iterative label propagation: initialize each vertex with its own ID, propagate minimum labels among neighbors, and repeat until convergence.

### 2.3 Parallelization Strategy

- **MPI**: Graph partitioned across ranks with edge-balanced distribution.
- **OpenCilk**: Within each rank, `cilk_for` parallelizes vertex processing with work-stealing.

# 3 Optimizations

## 3.1 Binary CSR Caching

We cache converted graphs as `.csrbin` files, reducing load times from minutes to seconds.

## 3.2 Graph Preprocessing

**Symmetrization** ensures undirected graphs; **degree-based reordering** improves cache locality.

## 3.3 Edge-Balanced Partitioning

Instead of simple vertex-block distribution, we balance edges per MPI rank, significantly improving load balance for scale-free graphs.

## 3.4 Asynchronous BFS

Our asynchronous variant overlaps computation with communication using double-buffered `MPI_Isend`, polls for messages during local propagation, and uses quiescence-based termination. This achieves **2.5x speedup** over synchronous.

## 3.5 64-bit Indexing

To support graphs with >2.14B edges, we use 64-bit `long long` for edge counts and row pointers.

# 4 Implementation

The project uses a Makefile with automatic detection of local OpenCilk and Matio library. SLURM scripts handle module loading and environment setup. Build artifacts are isolated per-test to prevent race conditions during parallel job submission.

# 5 Results

## 5.1 Test Environment

Experiments were conducted on the "Rome" partition (AMD EPYC, Rocky Linux 9, GCC 13.2.0, OpenMPI 5.0.3).

## 5.2 Performance Results

Results for `mawi_201512020330` (226M vertices, 480M edges):

| Configuration | Procs | Algorithm | Time (s) |
|---|---|---|---|
| Sequential Baseline | 1 | Sync | 7.56 |
| OpenCilk 10 Workers | 1 | Sync | 4.65 |
| Distributed Async | 8 | Async | **4.84** |

Table 1: Performance comparison. All find 3,971,144 components.

### 5.3 Scaling Analysis

#### 5.3.1 OpenCilk Scaling

| Workers | Time (s) | Speedup | Efficiency |
|---|---|---|---|
| 1 | 7.56 | 1.00× | 100% |
| 10 | 4.65 | 1.63× | 16.3% |

Table 2: Intra-node scaling (memory-bound workload).

#### 5.3.2 Async vs Sync

The asynchronous algorithm achieves **2.5x speedup** over synchronous on 8 MPI ranks by overlapping computation with communication and hiding network latency.

## 6 Conclusion

We implemented a hybrid MPI+OpenCilk parallel connected components algorithm with:

- Binary CSR caching for fast graph loading
- **Asynchronous BFS** providing 2.5x speedup
- Edge-balanced partitioning for scale-free graphs
- 64-bit indexing for billion-edge graphs

Future work: GPU acceleration and hybrid frontier-based approaches.